

# JullAca LimpIA

## Rutas de Limpieza Inteligentes

Documentación Técnica — Hackathon 2026  
Puno, Perú

Sistema de optimización de rutas de recolección de basura usando visión por computadora (YOLO-World) y algoritmos de ruteo inteligente. Convierte fotos ciudadanas en rutas de limpieza prioritarias.

## 1. Visión General del Sistema

El sistema resuelve el problema de las **rutas fijas de recolección de basura** que no responden a la demanda real, generando acumulación de residuos en zonas críticas, pérdida de tiempo y combustible.

**Flujo principal:**



- Reporte:** Ciudadano sube foto y marca ubicación en el mapa
- Análisis IA:** YOLO-World detecta objetos (contenedores, bolsas, desbordes) y calcula severidad
- Visualización:** Los reportes aparecen en el mapa con colores según nivel (Crítico/Medio/Bajo)

4. **Ruta Inteligente:** Algoritmo genera la ruta óptima priorizando puntos críticos por cercanía

## 2. Arquitectura del Sistema

| Componente     | Tecnología                          | Función                                     |
|----------------|-------------------------------------|---------------------------------------------|
| Frontend       | React 18 + Vite 5 + Leaflet         | Interfaz de usuario, mapas interactivos     |
| Backend IA     | FastAPI + YOLO-World + Ultralytics  | Detección de objetos y cálculo de severidad |
| Ruteo (OSRM)   | API pública router.project-osrm.org | Rutas por calles reales con OpenStreetMap   |
| Almacenamiento | localStorage del navegador          | Persistencia de reportes en el cliente      |

**Endpoints del Backend (puerto 8000):**

- GET /health — Health check
- GET /status — Estado del modelo y clases soportadas
- POST /analyze — Analiza imagen (devuelve JSON con detecciones + imagen anotada)
- POST /analyze-with-image — Analiza y devuelve imagen anotada como JPEG

## 3. Detección de Basura con YOLO-World

### 3.1 ¿Qué es YOLO-World?

YOLO-World es un modelo de detección de objetos **zero-shot** de última generación. A diferencia de los modelos tradicionales que solo detectan clases pre-entrenadas fijas (como COCO con 80 clases), YOLO-World permite **definir cualquier lista de clases en tiempo de inferencia** sin necesidad de re-entrenar. Esto lo hace ideal para detectar basura, ya que el vocabulario es muy variado.

### 3.2 Vocabulario de Detección

El sistema define **112 clases** en español e inglés organizadas en categorías:

| Categoría               | Ejemplos                                                          |
|-------------------------|-------------------------------------------------------------------|
| Contenedores            | waste container, dumpster, trash can, contenedor de basura        |
| Desbordes               | overflowing dumpster, full garbage container, overfilled dumpster |
| Bolsas                  | trash bags, garbage bags, bolsas de basura                        |
| Acumulaciones           | garbage pile, pile of trash, construction debris                  |
| Basura individual       | bottle, can, paper, plastic bag, tire, electronic waste           |
| Items callejeros (Perú) | maskara, vaso plastico, cascara, zapato, colchon                  |

### 3.3 Fallback Chain

El detector implementa una cadena de resiliencia:

- YOLO-World** (yolov8s-world.pt) — intenta cargar el modelo completo
- GPU/CUDA** — si hay GPU disponible, el modelo se mueve a CUDA para aceleración
- Frontend fallback** — si el backend no está disponible, el frontend usa un analizador simulado

### 3.4 Colores por Clase

Cada categoría de objeto detectado tiene un color asignado para los bounding boxes:

| Categoría         | Color (BGR)             | Visual                                                                              |
|-------------------|-------------------------|-------------------------------------------------------------------------------------|
| Contenedores      | Verde (60, 180, 75)     |  |
| Desbordes         | Rojo (60, 60, 255)      |  |
| Bolsas            | Naranja (0, 165, 255)   |  |
| Basura individual | Celeste (255, 180, 100) |  |

## 4. Cálculo de Severidad

El archivo `severity.py` implementa la lógica que convierte las detecciones del modelo en **niveles de severidad** accionables.

### 4.1 Clasificación de Objetos

| Grupo            | Nivel Asignado          | Ejemplos                                                |
|------------------|-------------------------|---------------------------------------------------------|
| CRITICAL_CLASSES | Crítico                 | overflowing dumpster, garbage pile, construction debris |
| MEDIUM_CLASSES   | Medio                   | trash bags, scattered trash, basura, residuos           |
| LOW_CLASSES      | Bajo                    | waste container, dumpster, recycling bin                |
| INDIVIDUAL_TRASH | Variable según cantidad | bottle, can, paper, tire, mascarilla                    |

### 4.2 Reglas de Severidad

El sistema aplica **9 reglas jerárquicas** en orden de prioridad:

| # | Condición                                            | Resultado |
|---|------------------------------------------------------|-----------|
| 1 | Desborde o acumulación grave presente                | Crítico   |
| 2 | Bolsas/basura en suelo + contenedor presente         | Medio     |
| 3 | 2+ objetos de basura individual (botellas, latas...) | Medio     |
| 4 | 2+ bolsas de basura sueltas                          | Medio     |
| 5 | Bolsas + basura individual combinados                | Medio     |
| 6 | Contenedor limpio + basura individual alrededor      | Medio     |
| 7 | Contenedor solo y limpio                             | Bajo      |
| 8 | 1-2 objetos individuales solos                       | Bajo      |
| 9 | Sin detecciones relevantes                           | Bajo      |

## 4.3 Métricas de Salida

El endpoint `POST /analyze` devuelve:

```
{
  "basura_detectada": true,
  "nivel": "Crítico",
  "confianza": "93%",
  "prioridad": "Alta",
  "resumen": "overflowing dumpster (93%), trash bags (87%)",
  "objetos": [{ "clase": "overflowing dumpster", "confianza": 0.93, "bbox": [120,45,300,280] }],
  "imagen_annotada_b64": "data:image/jpeg;base64,...",
  "colores_clases": { "overflowing dumpster": "#ff3c3c" }
}
```

## 5. Algoritmo de Ruta Inteligente

El algoritmo está implementado en `src/utils/routeOptimizer.js` y combina **ordenamiento por prioridad** con la heurística del **vecino más cercano**.

### 5.1 Lógica del Algoritmo

1. Asignar peso numérico a cada nivel:  
Crítico = 3, Medio = 2, Bajo = 1
2. Ordenar todos los reportes por prioridad descendente
3. Para cada grupo del mismo nivel de prioridad:
  - a. Empezar desde el punto actual (municipalidad primero)
  - b. Elegir el punto MÁS CERCANO al actual (distancia Haversine)
  - c. Avanzar a ese punto y repetir
4. Calcular métricas finales:
  - Distancia total (Haversine entre puntos consecutivos)
  - Tiempo estimado (velocidad promedio 30 km/h)
  - Combustible (0.4 L/km)
  - CO<sub>2</sub> evitado (2.68 kg/L de combustible)
  - Ahorro % vs ruta fija (22% más larga)

### 5.2 Distancia Haversine

Para calcular distancias entre coordenadas GPS se usa la **fórmula de Haversine**, que calcula la distancia del gran círculo entre dos puntos en una esfera (Tierra, R=6371 km):

$$d = 2R \cdot \arcsin(\sqrt{(\sin^2(\Delta\text{lat}/2) + \cos(\text{lat}_1) \cdot \cos(\text{lat}_2) \cdot \sin^2(\Delta\text{lng}/2))})$$

### 5.3 Ruteo por Calles Reales (OSRM)

Además de la ruta en línea recta, el sistema se conecta a la **API pública de OSRM** (Open Source Routing Machine, basada en OpenStreetMap) para obtener rutas que siguen **calles reales**:

- **URL:** `https://router.project-osrm.org/route/v1/driving/{coordenadas}?geometries=geojson`
- **Parámetros:** coordenadas en formato `lng, lat` separadas por punto y coma

- **Respuesta:** GeoJSON con la geometría de la ruta, distancia y duración

Si OSRM no está disponible (sin internet), el sistema **simula** la ruta generando puntos intermedios con desviaciones senoidales y aumentando la distancia un 25% sobre la línea recta.

**Nota:** OSRM es una API pública gratuita sin necesidad de API key, pero tiene límites de uso. Para producción, se recomienda hostear un servidor OSRM propio.

### 5.4 Métricas de Salida de la Ruta

| Métrica                 | Cálculo                                                      | Ejemplo |
|-------------------------|--------------------------------------------------------------|---------|
| Distancia               | Suma de Haversine entre cada parada                          | 12.4 km |
| Tiempo                  | $\text{Distancia} / 30 \text{ km/h} \times 60 \text{ min}$   | 25 min  |
| Combustible             | $\text{Distancia} \times 0.4 \text{ L/km}$                   | 5.0 L   |
| CO <sub>2</sub> evitado | $\text{Combustible} \times 2.68 \text{ kg/L}$                | 13.4 kg |
| Ahorro                  | $(\text{ruta\_fija} - \text{ruta\_opt}) / \text{ruta\_fija}$ | 18%     |



## 6. Flujo de Datos Completo

---

### 6.1 Reporte desde la App Ciudadana

```
Usuario (UserPage)
├─ Paso 1: Selecciona ubicación en el mapa (Leaflet)
├─ Paso 2: Toma/sube foto (cámara o galería)
├─ Paso 3: Describe el problema
└─ Guarda en localStorage → ReportsContext
    ↓
Aparece en el mapa de la vista de basura (BasuraPage)
    ↓
Botón "Procesar con IA" → POST /analyze → YOLO-World
    ↓
    Resultado: nivel, objetos, imagen anotada con bounding boxes
    ↓
Actualiza reporte en el mapa con colores según severidad
```

### 6.2 Ruta desde la Central de Monitoreo

```
Operador (BasuraPage)
├─ Ve todos los reportes en el mapa con colores
    Crítico · Medio · Bajo
├─ Presiona "Generar Ruta"
    ↓
    routeOptimizer.js ordena por prioridad + vecino cercano
    ↓
    OSRM calcula ruta por calles reales (o simulación offline)
    ↓
    Polyline dibuja la ruta en el mapa
    ↓
    Panel muestra: distancia, tiempo, combustible, CO2, ahorro
└─ Operador sigue el orden de paradas numeradas
```

## 6.3 Detalle del Análisis de Imagen



## 7. Estructura del Proyecto

```

rutas-limpieza-inteligentes/
├── ai-service/                                # Backend Python
│   ├── main.py                               # API FastAPI (entry point)
│   ├── detector.py                           # YOLO-World: carga, inferencia, dibujo
│   ├── severity.py                           # Lógica de severidad (9 reglas)
│   ├── requirements.txt                       # fastapi, uvicorn, ultralytics, Pillow
│   ├── run.sh                                # Script de inicio
│   └── yolov8s-world.pt                       # Modelo YOLO-World pre-entrenado
├── src/                                       # Frontend React
│   ├── main.jsx                              # Entry point
│   ├── App.jsx                               # Router (2 rutas: /user y /basura)
│   ├── styles.css                            # 1400+ líneas de estilos mobile-first
│   ├── pages/
│   │   ├── UserPage.jsx                     # App ciudadana (3 pasos: mapa, foto, desc)
│   │   └── BasuraPage.jsx                   # Dashboard operario (mapa + ruta + análisis)
│   ├── components/
│   │   ├── MapView.jsx                      # Mapa Leaflet con marcadores y polyline
│   │   ├── ReportForm.jsx                   # Formulario de reporte con análisis IA
│   │   ├── RoutePanel.jsx                   # Panel de ruta generada (métricas + paradas)
│   │   ├── StatsCards.jsx                   # Tarjetas de estadísticas
│   │   └── BboxOverlay.jsx                  # Overlay de bounding boxes en imagen
│   ├── context/
│   │   └── ReportsContext.jsx                # Estado global (useContext + localStorage)
│   ├── data/
│   │   └── mockReports.js                   # 7 reportes iniciales de Juliaca
│   ├── services/
│   │   └── aiAnalyzer.js                    # Cliente HTTP para backend IA
│   └── utils/
│       ├── distance.js                      # Fórmula Haversine
│       ├── routeOptimizer.js                # Algoritmo de ruta inteligente
│       ├── osrmRouter.js                    # Cliente OSRM + simulador offline
│       └── fakeImageAnalyzer.js              # Simulador de análisis (fallback)
├── vite.config.js
├── package.json
└── index.html
```

## 8. Tecnologías y Librerías

| Librería                 | Versión   | Propósito                              |
|--------------------------|-----------|----------------------------------------|
| React                    | 18.2      | Framework UI                           |
| Vite                     | 5.4       | Bundler y dev server                   |
| Leaflet / react-leaflet  | 1.9 / 4.2 | Mapas interactivos OpenStreetMap       |
| Lucide React             | 1.16      | Iconos SVG                             |
| react-router-dom         | 7.17      | Ruteo del frontend                     |
| FastAPI                  | 0.100+    | API REST del backend                   |
| Uvicorn                  | 0.23+     | Servidor ASGI                          |
| Ultralytics (YOLO-World) | 8.2+      | Detección zero-shot                    |
| Pillow                   | 10+       | Procesamiento de imágenes              |
| python-multipart         | 0.0.6+    | Soporte multipart/form-data en FastAPI |

## 9. Cómo Ejecutar

**Prerrequisitos:** Node.js 18+, Python 3.10+, pip

### Backend (IA)

```
cd rutas-limpieza-inteligentes/ai-service
pip install -r requirements.txt
python3 main.py
# → Servidor en http://localhost:8000
```

### Frontend

```
cd rutas-limpieza-inteligentes
npm install
```

```
npm run dev -- --host  
# → Servidor en http://localhost:5173
```

**Nota:** El modelo YOLO-World (~80 MB) se descarga automáticamente la primera vez desde la caché de Ultralytics. Si no hay GPU, se ejecuta en CPU. Sin backend, el frontend usa un analizador simulado.

## 10. Posibles Mejoras

| Área           | Mejora                                                                              |
|----------------|-------------------------------------------------------------------------------------|
| Backend        | Hostear OSRM propio para no depender de API pública con rate limits                 |
| Backend        | Implementar autenticación y base de datos (PostgreSQL + PostGIS)                    |
| IA             | Fine-tuning de YOLO-World con imágenes locales de contenedores peruanos             |
| Frontend       | Notificaciones push a camiones de basura con rutas asignadas                        |
| Frontend       | App móvil nativa (React Native) con GPS en tiempo real                              |
| Ruteo          | Algoritmo TSP (Traveling Salesman Problem) con restricciones de capacidad de camión |
| Almacenamiento | Migrar de localStorage a backend con API REST para datos persistentes               |

## 11. Resumen Visual del Sistema

